

Final Design Project Report:

Gesture-Control Hand

Team Fantastic 4:

Dice Jashnani, Jule Rohana, Zara Alom, & Zoe Obanu

Tuesday/Thursday 8:30 AM Section

5/19/2026

Front Matter

Abstract

Many people are unable to use a mouse due to limited hand mobility stemming from physical conditions, or they are older adults unfamiliar with technology, preventing a significant portion of the population from accessing basic internet. Our project's goal was to create a glove that lets someone navigate their computer using only intuitive hand gestures, no mouse, no keyboard, no prior tech experience required.

List of Figures

Figure 2.1: Touch sensors on the tips of the glove so that touch can be interpreted into PC inputs.

Figure 2.2: Controlling with other joints (wrist, shoulders, elbows, knees, toes)

Figure 2.3: Glove that confirms gestures only when a finger bend and thumb press occur simultaneously

Figure 2.4: Only uses fingers to control all functions (would require two gloves).

Figure 2.6: Decision Matrix

Figure 3.1.1: Glove Prototype 1-5

Figure 3.1.2: Glove Prototype 6

Figure 3.1.3: Glove Prototype 7

Figure 3.3.1: Wiring Housing Isometric View

Figure 3.3.2: Wiring Housing Front (View from computer-connecting side)

Figure 3.3.3: Wiring Housing Side view

Figure 3.3.4: Wiring Housing full drawing

Figure 3.4: Bill of Materials

Figure 3.5: Wiring Diagram

Figure 4.1: Goal Matrix

Table of Contents

Section 1 — Problem & Definition 1.1 Problem Statement 1.2 Objectives, Functions, and Constraints 1.3 Goals and Ranking 1.4 Benefits — Who, Why, Impact 1.5 Background and Technical Research 1.6 Intellectual Property Research Section 2 — Generate & Decide 2.1 Design Alternatives 2.2 Value Sensitive Design 2.3 Rank Order Comparison Matrix 2.4 Decision Matrix and Design Direction Section 3 — Implement & Iterate Section 3.1: Prototype Progression Section 3.2: Key Features of the Final Design Section 3.3: CAD and Engineering Drawings Section 3.4: Bill of Materials Section 3.5: Wiring and Electronics Section 3.6: Code	Section 4 — Evaluate Section 4.1: Goals Met --- How Do You Know? Section 4.2: What Worked and What Did Not Section 4.3: Actual vs. Intended Section 4.4: Next Steps Section 5 — Reflect 5.1 Computations and Technical Considerations 5.2 Ethical Considerations 5.3 Reflection on the Design Process References Appendices
--	--

Section 1 — Problem & Definition

1.1 Problem Statement

For many people, using a computer means using a mouse and keyboard, but those devices were not designed with everyone in mind. Deconstructing our problem through functional decomposition, the issue isn't that the concept of using a mouse is unknown; it is that there is no accessible option for individuals who are physically limited from using the mouse and the interactions that come with it. Individuals with limited hand mobility due to conditions such as arthritis, Parkinson's disease, or physical disability, as well as older adults who are unfamiliar with technology, find these standard input devices either painful or too intimidating to use. This presents a significant concern, as it prevents a substantial portion of the population from accessing the basic internet.

1.2 Objectives, Functions, and Constraints

The primary objective of this device is to ensure that the gestures required for operation come naturally to the user, eliminating the need for any prior technology experience. To achieve this, hand gestures must remain simple enough to accommodate individuals with limited hand mobility, who may struggle with complex motions. At the same time, the device must maintain sufficient reliability and consistency to support everyday use. Comfort is equally important, as the device should not cause extreme fatigue during extended periods of operation.

The device functions by detecting hand and finger positions through an array of flex sensors and touch sensors integrated throughout its structure. These detected gestures are then translated into standard PC inputs, including cursor movement and clicking. The overall design is intentionally kept simple, ensuring that the device remains easy to use for its intended users.

The primary constraints governing this design are budget and parts availability, as all components must be affordable and sourceable through the FYELIC lab. This is particularly relevant given that existing products on the market are prohibitively costly for many users. Time is an equally

significant constraint, as the complexity of the design is bounded by the length of the semester, meaning the scope of the project is limited to producing a functional prototype rather than a fully polished product.

1.3 Goals and Ranking

Our goals include having a flexible framework for easy addition of extra inputs (which means that the software is object-oriented around individual inputs so they're easy to add). We also want the sensors to be reliable, able to correctly identify a gesture 90% of the time. We want a contact sensor on the thumb with the same reliability, along with comprehensive auxiliary input functionality (One input for each connected finger flexed down, one input for contact sensor activation, at least one input for a combination of two fingers flexed down, and if time permits one input for each finger flexed up).

1.4 Benefits — Who, Why, Impact

The primary users are people who struggle to use a traditional keyboard/mouse setup, which primarily means the elderly and people with impaired hand movement. Secondary stakeholders include people who can easily use a keyboard/mouse setup, but use it for simple work and can switch to gesture control. This glove gives badly needed alternatives for an input system that most everyone uses.

1.5 Background and Technical Research

Technical Principles

The gesture-control glove relies on several core engineering principles. Flex sensors operate by changing their electrical resistance as they bend, meaning the degree of finger curvature can be measured as a variable voltage signal. Capacitive touch sensors detect contact by sensing a change in an electrical field when a conductive surface, such as a fingertip, comes near or touches them.

Because the Raspberry Pi operates digitally, an analog-to-digital converter (ADC) is required to translate the continuous voltage readings from the flex sensors into discrete values that the microcontroller can process. These processed signals are then mapped to standard computer inputs, allowing the glove to communicate with a PC and replicate the functions of a conventional mouse.

Existing solutions and their limitations

Product	Cost and Obtainability issues	Ease / Effectiveness of Use issues
Apple Head Pointer	Only on Apple Devices	Still needs full range of motion in hands
Myo Armband	Hard to set up for distinct users	Only has two inputs

Existing solutions tend to have limitations in relation to the control of the base device or issues with market adoption of the device: likely due to cost or ease-of-use constraints. Apple devices with inbuilt cameras include a feature known as Head Pointer, where users can enable it and use a vector (derived through the use of Computer Vision) projected perpendicular to their face to control the cursor. The cursor simply traces the movements made by the user's head, pointing where their head points. Because the motion of the cursor is proportional to the motion of the users head, neck strain can be induced by operations that require faster or erratic movements by the user. The user may have to move closer to the device to reduce the angle that they need to look, which in turn would induce eye strain. Furthermore, Apple Head Pointer only addresses injustices in control for the operations normally completed by a touch pad or mouse, meaning users must retain full range of motion in the hands to manipulate the keyboard (which would also control the clicking functions, in this case).

The Myo Armband is another example of technology that, while attempting to solve the issue of gesture control, tends to leave out detailed functions, only allowing for a few distinct inputs by the user. Electromyography is employed in a band that goes around the user's arm, sensing the clenching of muscles in the arm. When the user clenches their fist, it can be differentiated from when the user has an open hand. The armband would also need to be recalibrated to different users, as different humans employ muscles to different degrees with each action, adding more friction to the effective use of this product. Because this product has only two inputs, another input must be used for spatial control of the cursor or tactile control of the keyboard.

Potential Improvements

Most of the existing gloves and patents we viewed don't discriminate their inputs based on fingers, so they have a very limited input space. Moreover, much of the existing gloves have all of the mechanical and tech parts built on the glove, which makes it quite heavy and unwieldy to use, which is a big problem given the user base. Our design has a lot of the heavy parts wired to the glove rather than mounted on it, and it employs more sensors for a wider input space, which addresses these problems.

1.6 Intellectual Property Research

The first patent being assessed is for "Flexible smart glove," and is a utility patent filed under the number US9529433B2 in 2014 — meaning it is still active and expires in 2034. The patent is for a smart glove with electrical devices that detect fine finger movements and larger hand movements, and responds electrically. The patented glove has many touch sensors and pressure sensors, allowing for tactile control with minimal actual tactile feedback. This means users get intuitive and uninterrupted control. We plan to take inspiration from the placement of sensors on the glove.

The second patent being assessed is a Utility patent filed in 2016 for Devices for measuring finger motion and detection hand gestures. The device consists of loops for multiple fingers, joined in the center by cross-joint strips. The fingers together, when performing pinching or spreading motions, change the distance between them: this distance is measured and can be relayed to another device. This patent can provide for us a mechanical framework, mirroring the per-finger sensing needed for our own device. However, the sizing may pose a problem: the center joint-spanning band may have to have elastic of some form incorporated.

The third patent being assessed for design inspiration and data is titled “Control-Point Activation Condition Detection For Generating Corresponding Control Signals.” Filed in 2024, the patent describes a conductive path attached to a glove or user’s hand, incorporating RFID tags, cameras, and distance detection. It poses similar hardware to our gesture detection hand, though we plan to use flexure sensors instead of a conductive path. Most claims and technical hinge on the incorporation of a conductive path on the device, so we are free to use a lot of technical inspiration from the rest of the patent.

The first patent gives us good information in how to allow for seamless, fast, and intuitive control. The second shows us how to incorporate potential control inputs, such as pinch and zoom controls, and shows the clear merit in doing so. The third patent shows an arrangement and planning of electrical devices on the hand itself, allowing us to draw inspiration on the construction of the hand itself. These can be drawn together in the design of our own hand, but great care should be taken to not cross boundaries in relation to patented technology. Drawing these design concepts together allows us to make a product or concept that is heavily informed by the research of others, and supplements real movement the best.

Section 2 — Generate & Decide

2.1 Design Alternatives

Alternative 1 | Technique: Other People’s View

Children's finger painting inspired this design, as the actions in our formative years are very common and simple, allowing for ease of repetition. The pros of this design allow for easy and repetitive actions that someone using the device can do, even if their movement is partially impaired. The cons of this design include the potential to struggle to distinguish between intentional inputs and

accidental touches, since fingertips naturally make contact with surfaces during everyday hand movements, leading to false triggers and reduced reliability.

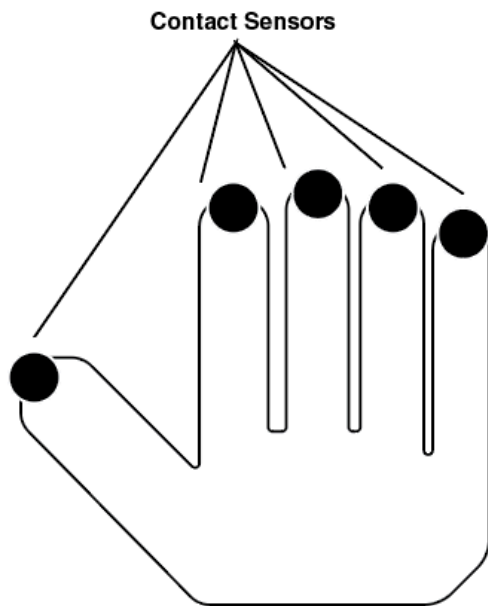


Figure 2.1: Touch sensors on the tips of the glove so that touch can be interpreted into PC inputs.

Alternative 2 | Technique: Scamper/Bionics

The design uses multiple joints of the body, including the wrists, shoulders, elbows, knees, and toes, as input points for the device, allowing a wide range of body movements to be mapped to PC inputs. Monkey's being able to hold items with their feet had inspired this design, as monkeys utilize all of their joints, and many joints we don't think of that could be easily used for our device. The pros of this design include a high degree of flexibility and input variety, as users can engage different parts of their body depending on their range of motion or physical ability. The cons include the complexity of tracking and interpreting so many joints simultaneously, which could make the system difficult to calibrate and prone to misreading unintended movements as inputs.

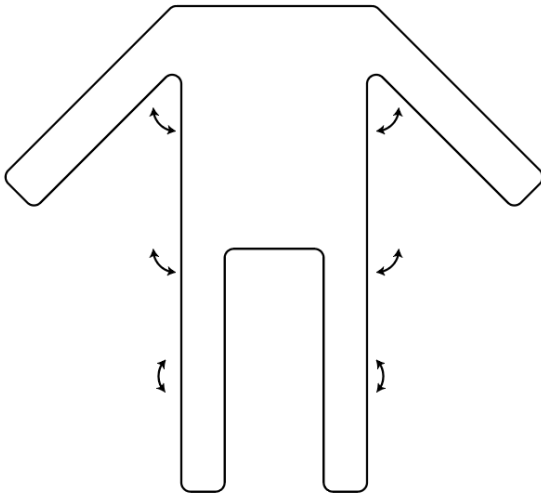


Figure 2.2: Controlling with other joints (wrist, shoulders, elbows, knees, toes)

Alternative 3 | Technique: Other People's View

This is a glove-based design that confirms gestures only when a finger bend and thumb press occur simultaneously, filtering out unintended movements caused by tremors or involuntary muscle activity. When put into the viewpoint of an individual with limited hand mobility, we thought of the ways the hand moves intuitively without the need for precise movements, but still accomplishes the general goal. The pros of this design include its ability to reduce false inputs significantly, making it far more reliable for users with conditions such as Parkinson's or general hand tremors. The cons include the requirement of some degree of coordinated movement between the fingers and thumb, which may still be a barrier for individuals with more severe mobility limitations.



Figure 2.3: Glove that confirms gestures only when a finger bend and thumb press occur simultaneously

Alternative 4 | Technique: Other people's view

This is a two-glove system that maps all PC input functions exclusively to finger movements, eliminating the need to raise or reposition the hand at any point. The left-hand thumb and pointer finger flex controls mouse movement left and right, while the right-hand thumb and pointer finger flex controls mouse movement up and down, with the left middle finger serving as a left click and the right middle finger serving as a right click. The pros of this design include its accessibility for users who have difficulty lifting or moving their arms, as all inputs are handled through small, low-effort finger gestures that keep the hands in a resting position. The cons include the reliance on two separate gloves working in coordination, which increases the complexity of the system and may present challenges in calibration, synchronization, and usability for individuals with asymmetric hand mobility.

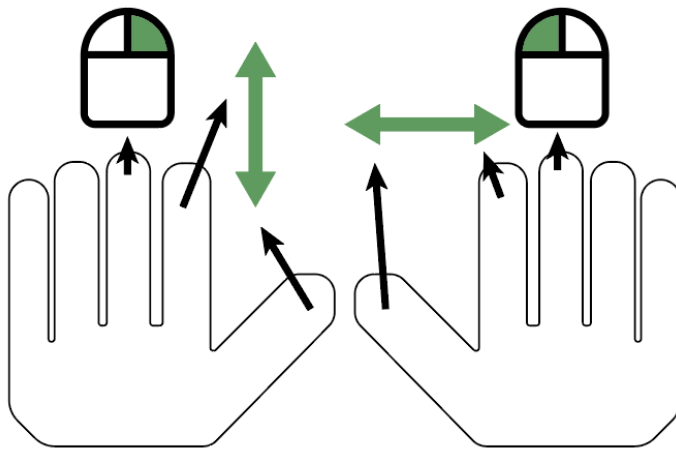


Figure 2.4: Only uses fingers to control all functions (would require two gloves). I.e., Left hand thumb/pointer flex -> move mouse left/right, Right hand thumb/pointer flex -> move mouse up/down, L Middle finger -> Left click, R Middle finger -> Right click. So the hand doesn't need to be raised

2.2 Value Sensitive Design

This design benefits users with limited fine motor control in their hands, since the simple, repetitive motions are intuitive and require minimal precision. However, it may exclude or disadvantage users with severe hand mobility limitations, tremors, or conditions that cause involuntary touch, as accidental inputs could be frequent. If the system misinterprets unintended touches, it could reduce user autonomy or create barriers rather than accessibility. This could push the design toward incorporating filtering mechanisms to better distinguish intentional input and ensure the system remains inclusive and dependable.

This design also benefits users with diverse physical abilities by offering multiple input points across the body, allowing individuals to rely on whichever joints they can control most effectively. However, it may exclude users with broader mobility impairments or fatigue-related conditions, as full-body interaction could be physically demanding or overwhelming. While flexibility

is high, the system could become difficult to learn, calibrate, or use consistently, potentially disadvantaging those who need simplicity. This suggests the design should emphasize customization and minimize cognitive and physical load to ensure it remains practical and equitable.

2.3 Rank Order Comparison

Ranking (least to most important): Weight (1x), Cost (2x), and Ease of Use (3x)

This ranking shows that the team prioritizes usability above all else, emphasizing that the device must be intuitive and accessible for new users. Cost is considered moderately important, while weight is the least critical factor as long as it remains within a reasonable limit. A slight surprise is that weight ranked lowest, meaning that we are willing to accept a heavier design if it significantly improves ease of use and affordability.

2.4 Decision Matrix and Design Direction

Figure 2.6: Decision Matrix

Alternative	Ease of Use (×3)	Cost (×2)	Weight (×1)	Total Score
Touch Sensor Glove	3 → 9	4 → 8	4 → 4	21
Multi-Joint Control	2 → 6	2 → 4	2 → 2	12
Gesture + Thumb Confirmation Glove	5 → 15	3 → 6	4 → 4	25

Alternative 3 scored the highest because it strongly prioritizes ease of use and reliability, which were weighted most heavily. The added gesture confirmation reduces accidental inputs, making it more practical for real users than Alternative 1, and less complex than Alternative 2. This result makes sense because a slightly more complex but reliable system is more valuable than a simple but error-prone one.

Design direction and next steps:

The design is heading toward a refined glove-based input system that emphasizes intentional gesture recognition, specifically using a combination of finger bending and thumb presses to confirm inputs. An initial prototype will be a simple glove with a few flex sensors and pressure sensors connected to a microcontroller to test gesture detection and filtering. From this early model, we expect to learn how accurately the system distinguishes intentional gestures from accidental movement, and how comfortable and intuitive the interactions feel for users, which will guide further improvements in sensitivity and usability.

Section 3 — Implement & Iterate

Section 3.1: Prototype Progression

An early prototype of the build included just sensors and the breadboard, providing a solid boilerplate to explore the potential for further technical development. This was at a stage when we were yet unsure exactly which sensors we wanted where, and having this boilerplate build helped assess the technical difficulty associated with different combinations of sensors. After the boilerplate came a better understanding of which sensors would go where, resulting in our current configuration. In the current prototype, some force sensors lacked functionality due to some solder joints snapping — likely due to the way some wires were routed. The joints are to be redone and wired with proper strain relief.



Figure 3.1.1: Glove Prototype 1-5

Our first glove demonstrates the different methods we used to attach the sensors and the placement of



the sensors.

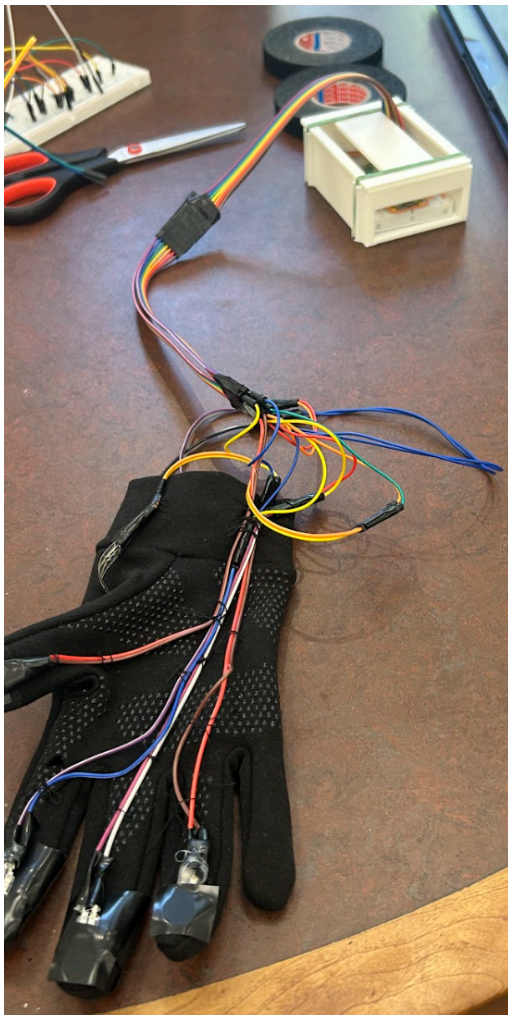


Figure 3.1.2: Glove Prototype 6

This is us implementing the wires fully and embedding them into the glove



Figure 3.1.3: Glove Prototype 7

This is our final design for the glove. We have the wires all embedded into the glove, and we sewed the glove to give a sleek look. We also braided the wires so they aren't going astray and look aesthetically pleasing.

Section 3.2: Key Features of the Final Design

The primary design attribute consists of flex sensors integrated into the glove's dorsal finger regions. This configuration specifically targets the 90% gesture recognition threshold and the requirement for independent finger-flex inputs. Technically, each sensor acts as a variable resistor where resistance scales with finger curvature; this change is captured via a voltage divider circuit. Since the Raspberry Pi lacks native analog support, an ADC converts these signals into discrete data. The software then evaluates these against calibrated thresholds to identify active flex states. This independent finger-tracking method significantly broadens the input range compared to the baseline systems discussed in Section 1.5.

A capacitive touch sensor integrated into the thumb pad functions as a precise contact mechanism, meeting the 90% accuracy objective for thumb-initiated inputs. This component detects field fluctuations triggered when a conductive fingertip completes a circuit at the pad. Unlike the variable flex sensors, this provides a binary output ideal for mouse-click emulation; triggers occur when the thumb meets another finger. Strategically placing the contact point on the thumb leverages

the anatomical fact that users with restricted mobility often maintain thumb control, ensuring the device remains accessible for our core demographic.

The software architecture utilizes an object-oriented framework designed to facilitate the rapid integration of new inputs. Every distinct gesture—whether a single flex, thumb contact, or multi-finger combination—is defined as a unique class with standardized methods for sensor polling and command dispatching. Consequently, expanding the gesture library merely involves registering a new class rather than modifying the core processing logic. This hierarchy also streamlines complex gestures; for example, a dual-finger input is a class that verifies the status of two pre-existing finger objects, effectively recycling detection logic instead of redundant coding.

Finally, the design isolates the main electronics from the wearable interface: sensors remain on the glove, while the Raspberry Pi and ADC reside in an external housing connected by flexible ribbon cabling. This separation fulfills the comfort requirements from Section 1.2 and enhances reliability by reducing glove mass, which prevents sensor drift during operation. In this setup, low-current signals are transmitted via cable to the off-board hub for signal processing and PC communication. This ensures the wearable component remains lightweight for those with minimal hand strength, addressing the specific ergonomic needs identified in Section 1.4.

Section 3.3: Wiring Housing CAD and Engineering Drawings

Figure 3.3.1: Isometric View

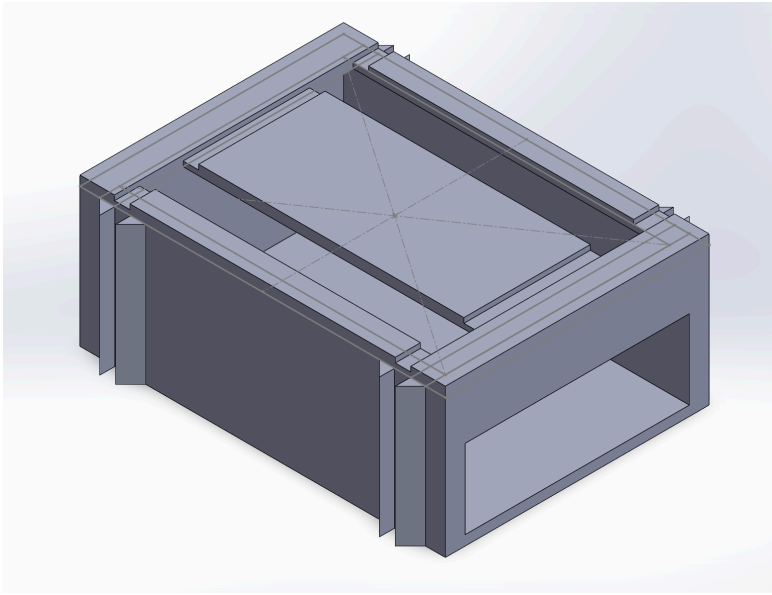


Figure 3.3.2: Front (View from computer-connecting side)



Figure 3.3.3: Side view

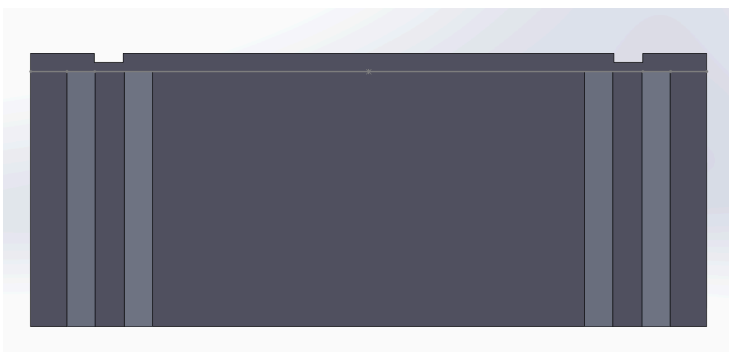
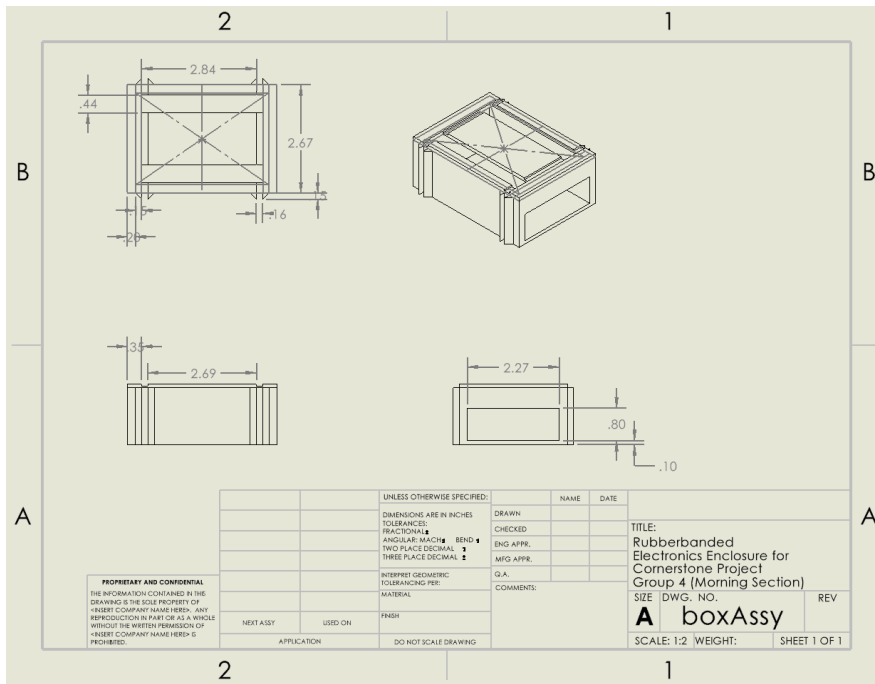


Figure 3,3,4: Full drawing



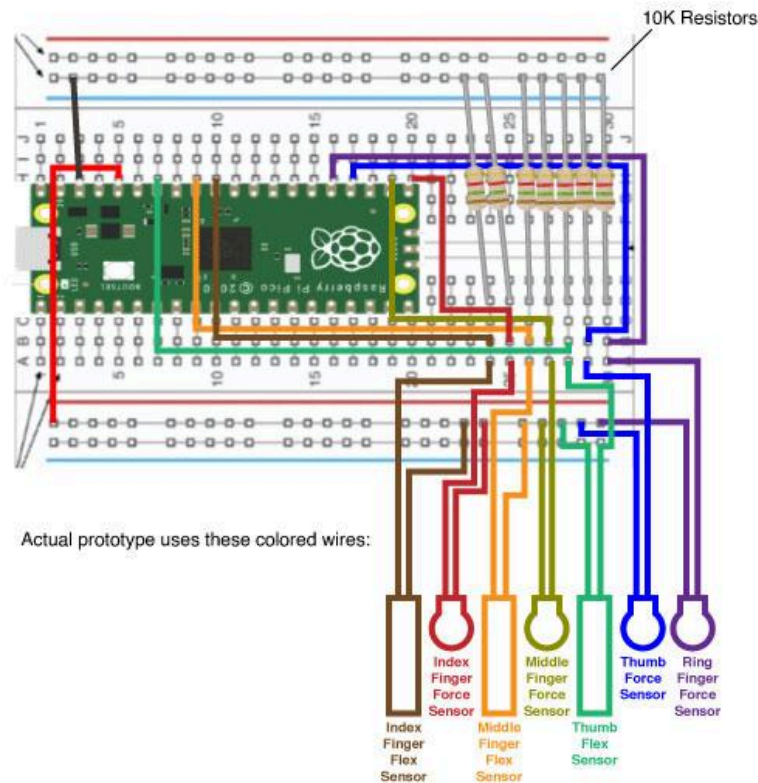
Section 3.4: Bill of Materials

Figure 3.4

Item #	Part Name	Description / Spec	Qty	Source	Unit Cost	Total Cost
1	Glove	Running gloves, black	1	Amazon	\$10/Unit (Bought as \$20/Pair)	\$20
2	Cloth Tape	Black	2	Amazon	\$3.25/Unit (Bought as \$6.50 for 2 rolls)	\$6.50
3	Flex Sensors	Resistor-based	3	FYELIC	N/A	N/A
4	Force Sensors	Resistor-Based	4	FYELIC	N/A	N/A
5	Black thread & needle	Used to sew in wires and cloth tape	1	Borrowed	N/A	N/A

Section 3.5: Wiring and Electronics

Figure 3.5: Wiring Diagram (rainbow wires are colored as such in the real prototype):



Section 3.6: Code

The code contains 3 Python files: `motions.py` (contains dictionaries featuring computer commands and the corresponding motions to activate them), `glove.py` (contains a class defining the main functions and variables of the glove), and `code.py` (contains the main loop for motion detection).

motions.py

`Motions.py` contains two dictionaries: `motions` and `motions_cursor`. The purpose of the dictionaries was to have an easily accessible and modifiable list of motions that would trigger certain actions on the computer. Both dictionaries are formatted with the key being the computer action and the value being the sensors whose value triggers the action.

Ex: `motions = {`

`"Left click": ["FORCE_THUMB"],`

There are two dictionaries because the glove has a default setting that controls basic keyboard and mouse functions, as well as a cursor setting that controls the cursor's position on the user's screen.

The motions dictionary contains all the motions and actions linked to the default setting, while the motions_cursor dictionary contains the motions and actions linked to the cursor setting.

glove.py

Glove.py contains the class Glove, which defines the readVals, calibrate, and completeAction functions, as well as individual action functions and other accessor and helper functions. The readVals function returns the values of the seven sensors on the glove. The three flex sensors return analog values based on how flexed the finger is at that moment, while the force sensors return digital values: they read 0 until force is applied, and they automatically read as 300. The calibrate function is intended to run while the user's hand is in a natural, unflexed position. It runs a loop that uses readVals to return the values of the flex sensors every 0.1 seconds for 5 seconds, and adds those values to a list corresponding to the sensor. It then runs a percentile function that returns the 30th percentile value of each list. Those values are regulated based on the sensitivity of each sensor and are then returned in a list called thresh to be used later to determine how an action triggers. Lastly, the completeAction function contains the actual logic for when a command is triggered. The function accepts a list of motions that the user completed and compares it to the values in either the motions dictionary or the motions_cursor dictionary, depending on the setting. If the motions in the initial list match a value in the dictionary, then the command that the motions are linked to is set as a variable in the function. This is followed by a series of if statements that lead to functions that trigger the command.

```
Ex:         if action == 'left click':

                self.leftClick()

        def leftClick(self):

            self.mouse.click(Mouse.LEFT_BUTTON)

            time.sleep(0.3) # debounce
```

code.py

Code.py contains the main Python loop for the program. It first runs the calibrate function from glove.py, and then it goes into the main loop. The main loop begins by reading the sensor values using the function in glove.py and by calculating the distance between the current value of the sensors and the max value of the sensor, in relation to the threshold value. If the flex sensor is 40% of the way to the max value, or the force sensor reads a value that is not 0, it enters “action mode,” meaning that any movements detected beyond that point will be added to a list. Once all sensors are back below the threshold, action mode is turned off, and the list of actions is sent into the completeAction function to trigger the computer command. This loop repeats until the glove is turned off.

Section 4 — Evaluate

Section 4.1: Goals Met --- How Do You Know?

Figure 4.1: Goal Matrix

Goal	Evidence	Result	Met?
Software supports a flexible sensor setup	The sensors are switched out with ease, and the code still functions	Sensors can all be tested easily	Met ▾
All sensors reliable	Tested multiple times and had a volt reader to see if the power is going through	Only the flex sensors are entirely reliable	Not Met ▾
Sensors self-calibrate	Tested with 5 classmates, and all had custom calibrations based on hand size and resting position	Allows the device to be used by an array of individuals	Met ▾

Section 4.2: What Worked and What Did Not

The code is fully functional and has been tested extensively with different sensor configurations to verify that its logic follows through correctly. During testing, we used a simple wired setup that made it straightforward to isolate whether any issue was coming from the code or the wiring, which significantly sped up the debugging process. Gesture detection, mode toggling, and action dispatching all performed as expected under these controlled conditions.

The primary recurring issue has been with the force sensors. In several cases, the sensors failed to register contact at all, either because the conductive surface could not make reliable contact with the glove material or because the press force required exceeded what users with limited mobility could comfortably apply. A separate but related problem was that the physical sensors themselves are quite fragile, with the metal contacts bending or breaking under normal use due to how thin and flimsy the material is. This caused some solder joints to fail, which we identified in Section 3.1, and those connections would need to be redone with proper strain relief before the glove could be used reliably for any extended period.

Section 4.3: Actual vs. Intended

The final prototype went through roughly five design iterations before reaching its current form, with most of the changes being driven by how the physical attachment of sensors affected both detection reliability and wearability. Early on, we used paper cutouts to imitate the sensors and hot glue to hold everything in place. This gave us a rough sense of the layout, but the result was messy, and when we swapped in the real sensors, the glove became noticeably bulky, making it difficult to move the fingers naturally. The added stiffness also meant the flex sensors could not bend freely, which directly hurt gesture detection accuracy. From there, we tried electrical tape and then cloth tape as separate alternatives, but neither solved the core problem. The sensors would shift around during use, leading to inconsistent readings that had nothing to do with actual hand movement. The solution we landed on was cutting into the glove material and sewing the sensors directly into it. This kept the profile thin, and the sensors stayed in place without adding bulk, which mattered for both usability and our accessibility goals, since a heavy or stiff glove is harder to use for someone with limited hand strength. The final prototype is meaningfully better than the original concept in this regard, though the iteration process made clear that the attachment method is just as important an engineering decision as sensor selection itself.

Section 4.4: Next Steps

The most immediate improvement would be expanding to a two-glove setup. A second glove would roughly double the available input space without requiring any new gesture vocabulary, since the object-oriented input framework from Section 3.2 could simply instantiate a parallel set of finger and thumb-contact objects for the second hand. Two gloves would also allow the system to distinguish left-hand and right-hand versions of the same gesture, enabling modifier-style inputs analogous to holding Shift or Ctrl, and addressing the limited input count we identified as a shortcoming of the Myo Armband in Section 1.5.

A second priority is adding more sensors, most notably an IMU such as the MPU6050 mounted on the back of the glove. Its accelerometer and gyroscope outputs would be read over I2C and fused to estimate hand orientation and angular velocity, feeding a new input class that interprets tilt as continuous scroll signals or wrist flicks as discrete gestures. Additional capacitive pads on the index, middle, and ring fingers would let the thumb-contact mechanism distinguish which finger is being touched, multiplying the number of distinct click-style inputs at minimal added weight.

The most significant functional gap in the current prototype is cursor movement itself, since flex sensors alone cannot capture spatial position. The IMU above is the most direct fix: integrating gyroscope readings over short time windows would yield relative hand motion to drive the cursor like a gyroscopic air mouse. Either this or a camera-based marker-tracking approach would need careful filtering so that small unintended tremors, common in users with conditions like Parkinson's disease, are suppressed rather than amplified.

With more time, validation would focus on testing with users from the actual target population. This means measuring whether the 90% reliability goal from Section 1.3 holds for users with arthritis or limited mobility, whose flex ranges differ from those of the team members who calibrated the prototype; a per-user calibration routine would almost certainly be needed. Several use cases also remain out of reach: text entry, precise drag-selection, and sustained use without recalibration as the glove shifts on the hand. Each would need to be addressed before the device could be considered a true mouse replacement.

Section 5 — Reflect

Section 5 is the one section you haven't written yet. It has three parts. Aim for about one page total. This section is graded on depth and honesty — not length.

5.1 Computations and Technical Considerations

Our primary constraint was the cost, although the final product ended up being far cheaper than expected. The glove came out to \$13.25/unit out of pocket, though the cost of the sensors, wires, raspberry pi pico, and all the sewing utility and cloth would likely bring the cost up to around \$25/unit. Our final future product also imagines the glove being used as part of a pair, so the pair would have a material cost of around \$50. This is still far cheaper than existing market sensor gloves (which range an order of magnitude higher), which is the closest mass-produced analog to our product.

5.2 Ethical Considerations

The device was designed for users with limited hand mobility, but the current prototype may unintentionally exclude the very people it was built for. The force sensors require a minimum press force that has already proved too demanding for some testers, as noted in Section 4.2, meaning a user with more severe limitations could find the device unusable entirely. This is a real contradiction we haven't fully resolved.

The calibration system introduces a related concern. Because thresholds are calculated from the user's own resting hand position, the device performs best for users whose flex range resembles the team members who built and tested it. Someone with arthritis may find their inputs miscalibrated in ways that aren't obvious, producing missed or false triggers without clear feedback. The people most likely to need the device are the least likely to have been represented in our testing.

On privacy, the glove processes everything locally and sends only standard mouse commands to the PC, so no biometric data is stored or transmitted. This is a meaningful advantage over camera-based alternatives.

The clearest safety concern is physical. Exposed solder joints and fragile sensor contacts can break under normal use, and for someone with limited hand control, a snapped wire becoming a sharp edge is a genuine injury risk. Proper strain relief is not a cosmetic fix; it is a baseline safety requirement before this device is used by anyone outside the team.

5.3 Reflection on the Design Process

By far the biggest takeaway is that the last 10% of a project can take half of the time. We spent about as many sessions fixing up the soldering as we did assembling the glove in the first place. Soldering was a new skill for most of us, so the soldered sensors snapped off fairly often, which was incredibly frustrating. However, it taught us that we have to consider the actual environment that a product is used in. We didn't consider that the sensors and wiring on the glove would undergo a lot of flexing, which led to a lot of breaks.

We also learned that documentation is incredibly important. We kept a documentation file from the first day of our project, and every time something of note happened, or we made changes to the plan, we would note it in the documentation. It was useful in writing this report, but also when we would be making the glove and needed to reference our wiring, coding quirks, or method of sewing.

Thus, our future projects will include documentation throughout the process, and we will build for more redundancy.

References

- M. L. R. Villalba, "ECE 4760 Sign Language Glove," *Copyright (C) 2013 Monica Lin, Roberto Villalba*.
https://people.ece.cornell.edu/land/courses/ece4760/FinalProjects/f2014/rdv28_mjl256/webpage/
- "Mister Gloves - a wireless USB gesture input system."
https://people.ece.cornell.edu/land/courses/ece4760/FinalProjects/s2010/ssc88_egl27/index.html
- "CaptoGlove. Wearable gaming, VR, AR, PC & mobile Controller.," *Kickstarter*, Feb. 02, 2020. <https://www.kickstarter.com/projects/146300353/captoglove>
- M. L. R. Villalba, "ECE 4760 Sign Language Glove," *Copyright (C) 2013 Monica Lin, Roberto Villalba*.
https://people.ece.cornell.edu/land/courses/ece4760/FinalProjects/f2014/rdv28_mjl256/webpage/
- "TactileGlove II - Sensor gloves for accurate hand force measurement — PPS," *PPS*.
<https://pressureprofile.com/body-pressure-mapping/tactile-glove>
- "Azure Kinect DK – Develop AI Models | Microsoft Azure."
<https://azure.microsoft.com/en-us/products/kinect-dk/>
- R. Smolin and R. Smolin, "Hand tracking and gesture recognition using AI: Applications and challenges," *Intellias*, May 04, 2026.
<https://intellias.com/hand-tracking-and-gesture-recognition-using-ai-applications-and-challenges/>
- "Gestures | Apple Developer Documentation," Apple Developer Documentation.
<https://developer.apple.com/design/human-interface-guidelines/gestures>
- Mendez, I., Hansen, B. W., Grabow, C. M., Smedegaard, E. J. L., Skogberg, N. B.,

Uth, X. J., Bruhn, A., Geng, B., & Kamavuako, E. N. (2017). Evaluation of the Myo armband for the classification of hand motions. IEEE ... International Conference on Rehabilitation Robotics : [proceedings], 2017, 1211–1214.

<https://doi.org/10.1109/ICORR.2017.8009414>

- “Getting Started with Raspberry Pi Pico and CircuitPython,” *Adafruit Learning System*, Jan. 21, 2021.
<https://learn.adafruit.com/getting-started-with-raspberry-pi-pico-circuitpython/circuitpython-programming-basics>
- “CircuitPython - Pico Download.” https://circuitpython.org/board/raspberry_pi_pico/
- MAINFRAME, “What’s the difference between digital and analog sensors?,” *YouTube*. Sep. 01, 2021. [Online]. Available:
<https://www.youtube.com/watch?v=xUeTkCffR8Y>
- [1]Soham Phanse, “ADC: 4 steps to Analog-to-Digital converters in Python!,” *Medium*, Feb. 16, 2025.
<https://medium.com/@soham.phanse/adc-4-steps-to-analog-to-digital-converters-in-python-7a271ca3b19c> (accessed May 17, 2026).
- “Flex Sensor Hookup Guide - SparkFun Learn.”
<https://learn.sparkfun.com/tutorials/flex-sensor-hookup-guide/all>
- “Raspberry Pi Pico GPIO Pinout.” <https://pico.pinout.xyz/>

Appendices

Access to our final code, files required for the Raspberry Pi Pico, and test code:

<https://github.com/varianig/Fantastic-Four-Glove>